

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call

```
zip_path = "/content/drive/MyDrive/PlantVillage/archive.zip"
```

```
import zipfile
import os

extract_path = "/content/dataset"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Extraction Done ✅")
```

Extraction Done ✅

```
import os

print(os.listdir("/content/dataset"))
```

```
['plantvillage', 'PlantVillage']
```

```
import os

print(len(os.listdir("/content/dataset/PlantVillage")))
print(len(os.listdir("/content/dataset/plantvillage")))
```

```
15
1
```

```
data_dir = "/content/dataset/PlantVillage"
```

```
import shutil
shutil.rmtree("/content/dataset/plantvillage")
print("Duplicate removed ✅")
```

Duplicate removed ✅

```
import os
import shutil
import random

src_dir = data_dir
dst_dir = "/content/small_dataset"

LIMIT_PER_CLASS = 200
```

```
os.makedirs(dst_dir, exist_ok=True)

for folder in os.listdir(src_dir):
    src_folder = os.path.join(src_dir, folder)
    dst_folder = os.path.join(dst_dir, folder)

    os.makedirs(dst_folder, exist_ok=True)

    images = os.listdir(src_folder)
    random.shuffle(images)

    selected = images[:LIMIT_PER_CLASS]

    for img in selected:
        shutil.copy(
            os.path.join(src_folder, img),
            os.path.join(dst_folder, img)
        )

print("Small dataset ready ✅")
```

Small dataset ready ✅

```
import os
import cv2
import numpy as np
from tqdm import tqdm

data_dir = "/content/small_dataset"

images = []
labels = []

IMG_SIZE = 128 # keep this fixed

for folder in os.listdir(data_dir):
    folder_path = os.path.join(data_dir, folder)

    for img_name in tqdm(os.listdir(folder_path)):
        img_path = os.path.join(folder_path, img_name)

        try:
            img = cv2.imread(img_path)
            img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

            images.append(gray)
            labels.append(folder)

        except:
            continue

print("Total Images Loaded:", len(images))
```

```

100%|██████████| 152/152 [00:00<00:00, 951.23it/s]
100%|██████████| 300/300 [00:00<00:00, 1025.84it/s]
100%|██████████| 361/361 [00:00<00:00, 778.04it/s]
100%|██████████| 372/372 [00:00<00:00, 468.70it/s]
100%|██████████| 358/358 [00:00<00:00, 361.32it/s]
100%|██████████| 371/371 [00:01<00:00, 284.50it/s]
100%|██████████| 388/388 [00:02<00:00, 165.40it/s]
100%|██████████| 392/392 [00:01<00:00, 364.24it/s]
100%|██████████| 354/354 [00:01<00:00, 291.96it/s]
100%|██████████| 375/375 [00:00<00:00, 429.83it/s]
100%|██████████| 375/375 [00:01<00:00, 307.07it/s]
100%|██████████| 377/377 [00:01<00:00, 340.94it/s]
100%|██████████| 365/365 [00:00<00:00, 790.47it/s]
100%|██████████| 371/371 [00:00<00:00, 759.55it/s]
100%|██████████| 360/360 [00:00<00:00, 807.22it/s]Total Images Loaded: 5271

```

```

from skimage.feature import hog

hog_features = []

for img in tqdm(images):
    features = hog(
        img,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        visualize=False
    )
    hog_features.append(features)

```

```

X = np.array(hog_features)
y = np.array(labels)

print("Feature shape:", X.shape)

```

```

100%|██████████| 5271/5271 [00:44<00:00, 117.89it/s]
Feature shape: (5271, 8100)

```

```

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_encoded = le.fit_transform(y)

print("Classes:", len(set(y_encoded)))

```

```
Classes: 15
```

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42
)

```

```
print("Train size:", len(X_train))  
print("Test size:", len(X_test))
```

```
Train size: 4216  
Test size: 1055
```

```
from sklearn.svm import SVC  
  
model = SVC(kernel='linear')  
  
model.fit(X_train, y_train)  
  
print("Model Training Done 
```

```
Model Training Done 
```

```
from sklearn.metrics import accuracy_score  
  
y_pred = model.predict(X_test)  
  
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.566824644549763
```

```
IMG_SIZE = 64
```

```
features = hog(  
    img,  
    orientations=9,  
    pixels_per_cell=(16, 16),  
    cells_per_block=(2, 2),  
    visualize=False  
)
```

```
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y_encoded, test_size=0.2, random_state=42  
)
```

```
from sklearn.svm import SVC  
  
model = SVC(kernel='rbf', C=10, gamma='scale')  
model.fit(X_train, y_train)
```

▼ SVC ⓘ ?

SVC(C=10)

```
from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)

print("New Accuracy:", accuracy_score(y_test, y_pred))
```